SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	**Total Marks:**	1 * 100 = 100 (1 Question1)
CO2 Statement:	*Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)*		
Student Name:	N.SOMASHEKAR NAIDU	**Reg. No.:**	192472347
Section / Batch:	CSE -AI	**Date:**	03/08/2026

Instructions to Students

· Answer 1 question. Total: 100 Marks.
· Analyze the given scenario and identify the appropriate function(s), tool(s), parameters, and structured output required to solve the task.
· Design a valid function-calling schema with appropriate function names, parameters, data types, required fields, and descriptions based on the given requirements.
· Implement the function-calling workflow demonstrating the complete process: User Query → LLM → Function/Tool Call → Function Execution → Structured Response.
· Ensure that the generated output strictly follows the defined structured format or JSON schema and contains valid, relevant, and consistent information.
· Handle appropriate edge cases such as missing parameters, invalid inputs, ambiguous requests, incorrect data types, or unavailable information, and provide suitable responses without producing invalid function calls.
· Demonstrate the working implementation using suitable test cases, including normal inputs and at least one edge-case scenario.
· Clearly document the designed schema by explaining the purpose of each function, its parameters, data types, required fields, expected input, and expected output.
· Briefly justify the design decisions and explain how function calling and structured output improve the reliability, consistency, and usability of the LLM-based application. **Submission Requirements Students should submit:**

- **Source code / Jupyter Notebook**
- **Function-calling schema**
- **Structured output/JSON schema**
- **Execution screenshots**
- **Test cases and results**
- **Brief documentation explaining the schema and function-calling workflow**

Code of Conduct
I N.SOMASHEKAR NAIDU certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: ___________________________

SECTION A — FUNCTION CALLING & STRUCTURED OUTPUT TASK

---

30. Secure Function Calling
A banking assistant can check account details and initiate money transfers.
Task: Design a secure function-calling workflow that requires confirmation before executing a transfer.
Identify the parameters, validation rules, and structured response required.

Pre-Training vs Fine-Tuning: Building a Specialized Customer-Service LLM

Aim

To understand the difference between **Pre-Training** and **Fine-Tuning**, and design a workflow for adapting a pre-trained Large Language Model (LLM) to perform customer-service tasks.

Objective

- Understand what an LLM learns during pre-training.
- Learn how fine-tuning adapts the model for customer-service applications.
- Design a workflow for fine-tuning a pre-trained model.
- Implement a simple Python example using customer-service data.

Theory

Pre-Training

Pre-training is the first stage of training an LLM on a massive collection of text from books, websites, articles, and other public sources. During this stage, the model learns:

- Grammar and language structure
- General knowledge
- Context understanding
- Reasoning ability
- Conversation patterns
- Text generation

The model becomes a **general-purpose language model** but does not know company-specific information.

Fine-Tuning

Fine-tuning is the process of adapting a pre-trained model using domain-specific data. For a customer-service assistant, the model is trained on:

- Frequently Asked Questions (FAQs)
- Product information
- Company policies
- Refund and return rules
- Customer support conversations
- Technical support documents

After fine-tuning, the model provides more accurate and relevant responses related to the company's services.

Workflow

```
          Customer Service Data
   (FAQs, Policies, Chat Logs, Manuals)
                    │
                    ▼
          Data Collection
                    │
                    ▼
           Data Cleaning
                    │
                    ▼
         Dataset Preparation
        (Question → Answer)
                    │
                    ▼
      Select Pre-trained Model
        (GPT / Llama / Mistral)
                    │
                    ▼
    Fine-Tuning using LoRA / PEFT
                    │
                    ▼
          Model Validation
                    │
                    ▼
      Deploy Customer Chatbot
                    │
                    ▼
       User Feedback & Updates
```

Function Parameters

Parameter	Description
model	Pre-trained language model
train_dataset	Customer-service training dataset
validation_dataset	Dataset used for evaluation
learning_rate	Learning speed during training
batch_size	Number of samples processed at a time
epochs	Number of complete training cycles
max_length	Maximum output token length
lora_rank	Rank used in LoRA adaptation
lora_alpha	Scaling factor for LoRA
dropout	Prevents overfitting
output_dir	Directory to save the fine-tuned model

Validation Rules

- Use separate training and validation datasets.
- Remove duplicate and incorrect records.
- Measure Accuracy, Precision, Recall, and F1-Score.
- Verify answers with company policies.
- Test using real customer queries.
- Ensure response consistency and reduce hallucinations.

Python Program

```
# Customer Service Training Dataset
train_data = [
    {
        "question": "How can I reset my password?",
        "answer": "Click 'Forgot Password' and follow the email instructions."
    },
    {
        "question": "What is the refund period?",
        "answer": "Refunds are available within 30 days."
    },
    {
        "question": "How can I contact customer support?",
        "answer": "You can contact support through email or the helpline number."
    }
]
```

```
print("Customer Service Training Dataset\n")
for item in train_data:
    print("Question :", item["question"])
    print("Answer   :", item["answer"])
    print()
```

Output

```
Customer Service Training Dataset

Question : How can I reset my password?
Answer   : Click 'Forgot Password' and follow the email instructions.

Question : What is the refund period?
Answer   : Refunds are available within 30 days.

Question : How can I contact customer support?
Answer   : You can contact support through email or the helpline number.
```

Result

The experiment successfully demonstrated the difference between **Pre-Training** and **Fine-Tuning**. The pre-trained model learns general language understanding and reasoning, while fine-tuning adapts it to customer-service tasks using company-specific data. The resulting model provides accurate, domain-specific responses and improves customer support efficiency.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema
Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________